

Full Stack Developer Take-Home Exercise

Overview

Build a small trading platform application that allows users to view and manage a trade blotter containing equity trades.

The application should consist of:

- A React frontend written in TypeScript
- A TypeScript backend API
- A database for persistence
- Real-time updates delivered to the UI
- Local execution via Docker and/or simple setup instructions

The exercise is intentionally open-ended. We are interested in your engineering decisions, architecture, code quality, and ability to deliver a coherent solution within the available time.

Business Context

You are building a simplified trade blotter for a broker.

A trade blotter is a real-time view of trades executed by traders and sales teams throughout the day.

Users should be able to:

- View trades
 - Create trades
 - Amend trades
 - Cancel trades
 - See updates reflected in real time
-

Core Requirements

Technology Requirements

Frontend

Must use:

- React
- TypeScript

Backend

Must use:

- TypeScript

Acceptable examples:

- Node.js + Express
- NestJS
- Fastify
- Hono
- deployed remotely (AWS, Azure, Heroku, etc)

Database

Use any database technology you feel is appropriate.

Examples:

- PostgreSQL
- SQLite
- MongoDB
- MySQL

Real Time

Live updates should be delivered using:

- WebSockets
- Socket.IO
- Server-Sent Events

Trade Model

At minimum, a trade should contain:

```
interface Trade {  
  id: string;  
  symbol: string;  
  quantity: number;  
  price: number;  
  side: "BUY" | "SELL";  
  trader: string;  
  tradeDate: string;  
  status: "ACTIVE" | "CANCELLED";  
}
```

You may extend this model if you wish.

Sample Trade Data

The following example shows the type of data consumed by the trade blotter. Candidates should be

able to generate and persist realistic randomized data using a similar structure.

```
[
  {
    "tradeId": "TRD-100001",
    "symbol": "AAPL",
    "side": "BUY",
    "quantity": 5000,
    "price": 227.45,
    "trader": "JSMITH",
    "book": "EQUITIES_UK",
    "counterparty": "Goldman Sachs",
    "tradeTimestamp": "2026-08-18T09:15:23Z",
    "status": "ACTIVE"
  },
  {
    "tradeId": "TRD-100002",
    "symbol": "MSFT",
    "side": "SELL",
    "quantity": 1200,
    "price": 534.22,
    "trader": "ABROWN",
    "book": "EQUITIES_US",
    "counterparty": "JP Morgan",
    "tradeTimestamp": "2026-08-18T09:18:54Z",
    "status": "ACTIVE"
  },
  {
    "tradeId": "TRD-100003",
    "symbol": "TSLA",
    "side": "BUY",
    "quantity": 800,
    "price": 341.75,
    "trader": "MJONES",
    "book": "TECH_GROWTH",
    "counterparty": "Morgan Stanley",
    "tradeTimestamp": "2026-08-18T09:20:11Z",
    "status": "CANCELLED"
  }
]
```

Suggested fields to randomize:

- tradeId
- symbol
- side
- quantity
- price
- trader
- book
- counterparty
- tradeTimestamp
- status

A dataset of approximately 100-1,000 trades is sufficient.

Optionally:

On application startup, if no data exists, the system should generate a realistic set of randomized trade data using the schema provided above.

Functional Requirements

Trade Blotter

Display a table/grid showing all trades.

The blotter should support:

- Sorting
 - Basic filtering
 - Refreshing data from the API
-

Create Trade

Users should be able to create a trade.

Validation should be applied where appropriate.

Amend Trade

Users should be able to update an existing trade.

Cancel Trade

Users should be able to cancel a trade.

We are not looking for complex regulatory workflows.

A simple status transition is sufficient.

Live Updates

Changes made by one client should be visible to all connected clients without requiring a page refresh.

This can be demonstrated through:

- WebSockets
- Socket.IO
- Server-Sent Events

Non-Functional Requirements

We expect:

- Clean code
- unit tests
- Sensible architecture
- Appropriate use of TypeScript
- Reasonable error handling
- Automated testing where appropriate
- Thoughtful UX

We do not expect production-ready systems.

AI Usage

AI-assisted development is encouraged.

However, we would like to understand how AI tools were used during development.

Please include:

AI Usage Report

A short document describing:

- Which AI tools were used
- How they were used
- Examples of prompts
- Key architectural or implementation decisions influenced by AI
- Areas where you accepted or rejected AI-generated suggestions

Prompt Log

Please include a file containing:

- Significant prompts used during development
- Relevant responses (summarised if necessary)

Example:

```
Prompt:
"Generate a WebSocket architecture for a React and Express application"

Outcome:
Used initial design but replaced Socket.IO with native WebSockets due to
```

simplicity.

We are not interested in exhaustive logs; a representative sample is sufficient.

Deliverables

Provide:

```
README.md
frontend/
backend/
database/
docker-compose.yml (optional but preferred)
```

The README should explain:

- Architecture decisions
 - Installation instructions
 - How to run the application
 - How to run tests
 - Assumptions made
 - Trade-offs accepted
-

Bonus Ideas (Optional)

Candidates are not expected to complete all of these.

Possible extensions:

Audit Trail

Maintain a history of trade amendments.

Position Summary

Calculate net positions by symbol.

P&L View

Display aggregate P&L by symbol.

User Authentication

Simple login capability.

Trade Validation Rules

Examples:

- Quantity must be positive
- Price must be positive

- Trader name required

Virtualised Grids

Use AG Grid, TanStack Table, or similar.

Containerisation

Provide Docker support.

Deployment

Deploy the application to a cloud provider or give us a Git repository we can install and run locally.

It must be OS agnostic (we may preview on Windows, Linux or Mac)

Assessment Criteria

Engineering Quality (30%)

- Code quality
- Project structure
- Maintainability

TypeScript Usage (20%)

- Type safety
- Domain modelling
- API contracts

Full Stack Design (20%)

- Frontend/backend interaction
- Database design
- API design

User Experience (10%)

- Usability
- Responsiveness
- Error handling

Testing (10%)

- Unit and integration testing
- Sensible coverage

Communication (10%)

- README quality

- AI usage report
 - Explanation of decisions
-

Expected Time Commitment

This exercise is designed to take approximately **8-15 hours** and should be completed within **7 calendar days** from receipt.

We do not expect every possible feature to be implemented. We value thoughtful trade-offs and clear communication more than feature completeness.
